

TaskFlow

Gerenciador de Projetos Acadêmicos

Desenvolvimento Web — CC5M

Arthur Vieira · Rafael Fassina · Thiago Gama

1. Descrição do Produto

O **TaskFlow** é uma aplicação web para gerenciamento de projetos acadêmicos colaborativos. Equipes de estudantes podem criar projetos, convidar membros, criar tarefas e acompanhar o progresso de cada atividade em um quadro Kanban com três colunas: *A fazer*, *Em andamento* e *Concluída*.

Funcionalidades

- **Autenticação:** cadastro, login e logout com senha hashada via bcrypt.
- **Projetos:** criação, visualização e exclusão de projetos próprios.
- **Membros:** adição e remoção de membros por e-mail (somente o dono pode gerenciar).
- **Tarefas:** criação, edição, exclusão e atualização de status por qualquer membro.
- **Atribuição:** cada tarefa pode ser atribuída a um membro específico do projeto.
- **Comentários:** membros podem comentar em tarefas para registrar atualizações e discussões.
- **Histórico de atividades:** cada projeto mantém um log das ações relevantes (criação de tarefas, adição de membros, etc.).
- **Autorização:** rotas protegidas; operações destrutivas restritas ao dono do projeto.

2. Modelo de Dados (Diagrama ER)

users	
PK id	uint
name	varchar NOT NULL
email	varchar UNIQUE NOT NULL
password	varchar NOT NULL
created_at	timestampz
updated_at	timestampz
deleted_at	timestampz (soft delete)

projects	
PK id	uint
title	varchar NOT NULL
description	text
FK owner_id	→ users.id
created_at	timestampz
updated_at	timestampz
deleted_at	timestampz (soft delete)

← owner
← assignee
N:M ↕
project_members

tasks	
PK id	uint
title	varchar NOT NULL
description	text
status	varchar DEFAULT 'todo'
FK project_id	→ projects.id
FK assignee_id	→ users.id (nullable)
created_at	timestampz
updated_at	timestampz
deleted_at	timestampz (soft delete)

← 1:N

project_members (tabela de junção N:M)	comments
FK project_id → projects.id	PK id uint
FK user_id → users.id	body text NOT NULL
	FK task_id → tasks.id
	FK user_id → users.id
	created_at timestamptz
	updated_at timestamptz
	deleted_at timestamptz (soft delete)

activity_logs
PK id uint
action text NOT NULL
FK project_id → projects.id
FK user_id → users.id
created_at timestamptz
updated_at timestamptz
deleted_at timestamptz (soft delete)

O GORM gerencia `deleted_at` automaticamente (soft delete): registros excluídos não são removidos fisicamente, apenas marcados com um timestamp, preservando integridade referencial.

3. Arquitetura da Aplicação

O projeto segue o padrão **Application Factory** e organização em camadas análoga ao Flask com Blueprints, adaptada para Go/Gin:

```

taskflow/ └─ cmd/ │ └─ server/ ← ponto de entrada (main.go) │ └─ seed/ ←
script de dados iniciais └─ internal/ │ └─ app/ ← Application Factory
(NewApp) │ └─ config/ ← leitura de variáveis de ambiente │ └─ database/ ←
conexão e AutoMigrate │ └─ models/ ← User, Project, Task, Comment, ActivityLog
(GORM) │ └─ repositories/ ← interfaces + implementações GORM │ └─ handlers/ ←
lógica HTTP por domínio │ └─ routes/ ← registro de rotas ("Blueprints") │ └─
middleware/ ← autenticação via sessão └─ templates/ │ └─ layout/base.html ←
layout base Bootstrap │ └─ auth/ projects/ tasks/ └─ static/css/ ← CSS
customizado └─ tests/ ← testes com mocks (sem banco real) └─ docker-
compose.yml

```

Fluxo de uma requisição

```
HTTP Request → gin.Engine → Middleware: LoadUser() (injeta current_user no contexto) → Middleware: Required() (bloqueia rotas protegidas) → Handler (ex: ProjectHandler.Show) → Repository (ex: projectRepo.WithMembers) → GORM → PostgreSQL ← modelo preenchido ← c.HTML(200, "projects/show", gin.H{...}) → htmlRenderer.Instance("projects/show", data) → template.Lookup("base").Execute(w, data) ← HTTP Response (HTML)
```

Correspondência com os critérios da disciplina

Critério	Implementação
Application Factory	<code>internal/app/app.go</code> — <code>NewApp(cfg)</code> centraliza toda inicialização
Blueprints	<code>internal/routes/auth.go</code> , <code>projects.go</code> , <code>tasks.go</code>
Forms / Validação	Structs com tags <code>binding:"required,min= ... "</code> via Gin + <code>ShouldBind</code>
Templates	Herança com <code>base.html</code> e blocos <code>{{block "content"}}</code>
Testes	21 testes em <code>tests/</code> usando mocks sem banco real
ORM	GORM com <code>AutoMigrate</code> , relacionamentos, <code>Preload</code> e <code>soft delete</code>
Boas Práticas	Repository Pattern, injeção de dependência, configuração por env vars

4. Detalhes Técnicos e Trechos de Código

4.1 Application Factory

Toda inicialização ocorre em `NewApp`: banco, migrações, repositórios, handlers, middlewares e rotas. Isso permite instanciar a aplicação em testes sem subir o servidor.

```

func NewApp(cfg *config.Config) (*App, error) {
    gin.SetMode(cfg.GinMode)

    db, err := database.Connect(cfg.DatabaseURL)
    if err != nil { return nil, err }
    database.Migrate(db)

    userRepo      := repositories.NewUserRepository(db)
    projectRepo    := repositories.NewProjectRepository(db)
    taskRepo       := repositories.NewTaskRepository(db)
    activityRepo   := repositories.NewActivityRepository(db)

    authHandler    := handlers.NewAuthHandler(userRepo)
    projectHandler  := handlers.NewProjectHandler(projectRepo, userRepo, activityRepo)
    taskHandler     := handlers.NewTaskHandler(taskRepo, projectRepo, userRepo, activityRepo)

    r := gin.Default()
    r.Use(sessions.Sessions("taskflow_session", cookie.NewStore( ... )))
    r.Use(middleware.NewAuthMiddleware(userRepo).LoadUser())
    r.HTMLRender = buildRenderer()

    routes.RegisterAuth(r, authHandler)
    routes.RegisterProjects(r, projectHandler)
    routes.RegisterTasks(r, taskHandler)

    return &App{Router: r, DB: db}, nil
}

```

4.2 Repository Pattern com Interfaces

Cada repositório expõe uma interface; os handlers dependem da interface, não da implementação GORM. Isso permite substituir a implementação por mocks nos testes.

```
// interfaces.go
type ProjectRepository interface {
    Create(project *models.Project) error
    FindByID(id uint) (*models.Project, error)
    WithMembers(id uint) (*models.Project, error)
    FindByMemberID(userID uint) ([]models.Project, error)
    AddMember(projectID, userID uint) error
    RemoveMember(projectID, userID uint) error
    IsMember(projectID, userID uint) bool
    Delete(id uint) error
}

// project_repository.go - implementação GORM
func (r *projectRepo) WithMembers(id uint) (*models.Project, error) {
    var p models.Project
    err := r.db.
        Preload("Owner").
        Preload("Members").
        Preload("Tasks").
        Preload("Tasks.Assignee").
        First(&p, id).Error
    return &p, err
}
```

4.3 Validação de Formulários

Gin usa struct tags para validar dados do formulário antes de chegar à lógica de negócio:

```
type RegisterInput struct {
    Name          string `form:"name"           binding:"required,min=2,max=100"`
    Email         string `form:"email"          binding:"required,email"`
    Password      string `form:"password"       binding:"required,min=6"`
    PasswordConfirm string `form:"password_confirm" binding:"required"`
}

func (h *AuthHandler) Register(c *gin.Context) {
    var input RegisterInput
    if err := c.ShouldBind(&input); err != nil {
        c.HTML(422, "register", gin.H{"Error": "Preencha todos os campos corretamente."})
        return
    }
    if input.Password != input.PasswordConfirm {
        c.HTML(422, "register", gin.H{"Error": "As senhas não coincidem."})
        return
    }
    // ...
}
```

4.4 Autenticação e Middleware

Dois middlewares compõem o sistema de autenticação:

```
// LoadUser - global, carrega o usuário se logado (não bloqueia anônimos)
func (m *AuthMiddleware) LoadUser() gin.HandlerFunc {
    return func(c *gin.Context) {
        session := sessions.Default(c)
        if id, ok := session.Get("user_id").(int); ok {
            if user, err := m.userRepo.FindByID(uint(id)); err == nil {
                c.Set("current_user", user)
            }
        }
        c.Next()
    }
}

// Required - aplicado por rota, redireciona não autenticados
func Required() gin.HandlerFunc {
    return func(c *gin.Context) {
        if _, exists := c.Get("current_user"); !exists {
            c.Redirect(302, "/auth/login")
            c.Abort()
        }
        c.Next()
    }
}
```

4.5 Herança de Templates

O layout base define blocos substituíveis. Cada página herda o layout e preenche os blocos:


```

<!-- templates/layout/base.html -->
{{define "base"}}
<!DOCTYPE html><html lang="pt-BR">
<head>
  <title>{{block "title" .}}TaskFlow{{end}}</title>
  <link href="bootstrap@5.3.3/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
  <nav class="navbar navbar-dark bg-primary">... </nav>
  <main class="py-4">{{block "content" .}}{{end}}</main>
  <script src="bootstrap@5.3.3/js/bootstrap.bundle.min.js"></script>
  {{block "scripts" .}}{{end}}
</body></html>
{{end}}

<!-- templates/auth/login.html -->
{{define "login"}}{{template "base" .}}{{end}}
{{define "title"}}Login{{end}}
{{define "content"}}
<div class="container">
  <!-- formulário de login -->
</div>
{{end}}

```

4.6 Quadro Kanban por Status

O handler agrupa as tarefas por status antes de passar ao template:

```

func (h *ProjectHandler) Show(c *gin.Context) {
    project, err := h.projectRepo.WithMembers(id)
    if err != nil {
        c.HTML(404, "error", gin.H{"Error": "Projeto não encontrado."})
        return
    }
    activities, _ := h.activityRepo.FindByProjectID(id, 20)
    c.HTML(200, "projects/show", gin.H{
        "Todo":      filterByStatus(project.Tasks, "todo"),
        "InProgress": filterByStatus(project.Tasks, "in_progress"),
        "Done":       filterByStatus(project.Tasks, "done"),
        "Activities": activities,
    })
}

func filterByStatus(tasks []models.Task, status string) []models.Task {
    var out []models.Task
    for _, t := range tasks {
        if t.Status == status { out = append(out, t) }
    }
    return out
}

```

5. Tecnologias Utilizadas e Justificativa

Tecnologia	Versão	Justificativa
Go	1.25	Compilado, tipado, concorrência nativa, binário único para deploy. Excelente para servidores HTTP de alta performance.
Gin	1.12	Framework HTTP minimalista e rápido para Go. Oferece roteamento, middleware, binding de formulários e renderização de templates de forma integrada.
GORM	1.25	ORM maduro para Go com suporte a migrations automáticas, relacionamentos, soft delete e Preload de associações — reduz código repetitivo de SQL.
PostgreSQL	16	Banco relacional robusto, open-source, com suporte completo a constraints, transações e ACID. Ideal para dados relacionais com integridade referencial.
Bootstrap	5.3.3	Framework CSS responsivo e completo. Permite criar uma interface funcional e estética rapidamente via CDN, sem build pipeline de frontend.
gin-contrib/sessions	1.0.1	Gerenciamento de sessão via cookie assinado. Simples, sem necessidade de armazenamento externo (Redis/Memcached).
bcrypt	—	Algoritmo de hash adaptativo para senhas. Resistente a ataques de força bruta por ser computacionalmente custoso por design.
Docker Compose	—	Orquestração local de app + banco em um único comando, eliminando configuração manual do ambiente de desenvolvimento.

6. Como Executar o Projeto

Pré-requisitos

- Go 1.25+ instalado
- Docker e Docker Compose instalados

Opção A — Desenvolvimento local (recomendado)

```
# 1. Clonar o repositório
git clone <url-do-repositorio>
cd taskflow

# 2. Criar arquivo de configuração
cp .env.example .env

# 3. Subir apenas o banco de dados via Docker
docker-compose up -d db

# 4. Instalar dependências Go
go mod tidy

# 5. Executar a aplicação
go run ./cmd/server

# Acesse: http://localhost:3000
```

Opção B — Docker Compose completo

```
# Subir app + banco juntos
docker-compose up --build

# Parar tudo
docker-compose down

# Parar e remover volumes (limpar banco)
docker-compose down -v
```

Variáveis de ambiente (.env)

Variável	Padrão	Descrição
DATABASE_URL	postgres://taskflow:taskflow@localhost:5432/taskflow	String de conexão PostgreSQL
SESSION_SECRET	dev-secret	Chave para assinar cookies de sessão
PORT	3000	Porta HTTP do servidor
GIN_MODE	debug	debug ou release

Executar os testes

```
# Todos os testes (sem banco - usa mocks em memória)
go test ./tests/ ... -v

# Via Makefile
make test
```

Os testes não dependem do banco de dados. Repositórios são substituídos por implementações em memória (mocks), garantindo execução isolada e rápida. **21 testes passando.**

7. Cobertura de Testes

Os testes estão em `tests/` e usam *mocks* que implementam as mesmas interfaces dos repositórios reais. Isso permite testar toda a lógica HTTP sem banco de dados.

Arquivo	Testes	O que cobre
<code>auth_test.go</code>	8	Login/registro correto e com erros, senha inválida, e-mail duplicado, logout
<code>project_test.go</code>	7	Proteção de rotas, criação válida/inválida, membros, exclusão
<code>task_test.go</code>	6	Criação, atualização de status, validação de enum, exclusão, proteção de rotas

Exemplo de teste

```
func TestCreateProjectWithValidDataRedirects(t *testing.T) {
    env := newTestEnv(t)
    cookie := loginAndGetCookie(t, env, "criador@test.com", "senha123")

    form := url.Values{
        "title":      {"Trabalho de Redes"},
        "description": {"Projeto para a disciplina"},
    }
    req := httptest.NewRequest(http.MethodPost, "/projects",
        strings.NewReader(form.Encode()))
    req.Header.Set("Content-Type", "application/x-www-form-urlencoded")
    req.AddCookie(cookie)

    w := httptest.NewRecorder()
    env.router.ServeHTTP(w, req)

    if w.Code != http.StatusFound {
        t.Errorf("esperado redirect 302, obteve %d", w.Code)
    }
    projects, _ := env.projectRepo.FindByMemberID(1)
    if len(projects) == 0 {
        t.Error("projeto deveria ter sido criado")
    }
}
```

8. Principais Desafios e Soluções

Desafio 1 — Herança de templates em Go

O Go não possui herança de templates nativa como Jinja2/Django. A biblioteca `html/template` usa blocos (`{{block}}` / `{{define}}`), mas a integração com o Gin exige cuidado na forma como os templates são registrados.

Problema encontrado: ao usar `multitemplate.AddFromFiles("login", base, login.html)`, o `ParseFiles` nomeia templates pelo *nome do arquivo* (ex: `"login.html"`), não pelo nome passado para `New("login")`. O template raiz ficava com `Tree = nil` → resposta HTTP 200 com corpo vazio (página em branco).

Solução: implementar um renderer customizado que, ao renderizar, chama `r[name].Lookup("base")` — o template `"base"` tem sua árvore corretamente populada pelo `ParseFiles` e compartilha o mesmo namespace com os blocos da página (`"content"`, `"title"`), resolvendo-os na hora da execução.

```
type htmlRenderer map[string]*template.Template

func (r htmlRenderer) Instance(name string, data any) render.Render {
    return render.HTML{
        Template: r[name].Lookup("base"), // ← solução
        Data:     data,
    }
}
```

Desafio 2 — Testes sem banco de dados

Testes de integração que dependem de banco são lentos, frágeis e difíceis de isolar. A solução foi criar repositórios mock em memória (`tests/mocks.go`) que implementam as mesmas interfaces (`UserRepository` , `ProjectRepository` , `TaskRepository`). Como os handlers dependem de interfaces (não de structs GORM), a substituição é transparente.

Desafio 3 — Autorização granular

A regra de negócio exige distinção entre *membro* (pode criar/editar tarefas) e *dono* (pode deletar projeto, gerenciar membros). A solução foi verificar `project.OwnerID == user.ID` nas operações restritas, separada da verificação de membros `IsMember(projectID, userID)` , evitando lógica duplicada.

Desafio 4 — Configuração de ambiente

Hardcodar a string de conexão impossibilita a mudança entre ambientes. A solução foi centralizar toda configuração no pacote `internal/config` com leitura de variáveis de ambiente e fallback para valores de desenvolvimento, integrando com `godotenv` para carregar um arquivo `.env` local.